

Illumio Agentless K8s Segmentation: Hybrid Multi-Cloud Platform Requirements

Document Purpose

This document defines requirements for the Illumio Agentless Kubernetes Segmentation solution for Hybrid Multi-Cloud customer environments aligning. It describes how Kubernetes workloads, labels, traffic, policy, and enforcement should integrate into the Illumio platform's hybrid multi-cloud workflows so that customers have a unified experience across their entire infrastructure — Kubernetes, cloud, and on-premises data centers.

Why Platform Alignment Matters

Aligning agentless K8s segmentation with the Illumio platform provides several strategic advantages:

- **Single product, single experience** — customers invest in one platform to segment their entire hybrid multi-cloud environment — Kubernetes, cloud, and on-premises workloads. Aligning the agentless K8s offering with the existing platform ensures that customers perceive and purchase a single, cohesive product — not a set of disconnected solutions. This strengthens the sales motion by demonstrating that Illumio provides consistent segmentation regardless of where workloads run.
 - **Single policy model** — customers define security intent once, and it applies across all workload types. No need to author, maintain, or reconcile separate policies for each environment.
 - **Hybrid visibility** — traffic between K8s workloads and non-K8s infrastructure is visible in the Unified map, enabling security teams to understand cross-environment dependencies without switching tools.
-
- **Operational simplicity** — security teams work in one console with one set of workflows, regardless of the underlying infrastructure. This extends beyond the UI — a single platform means a single API for policies, automation and integration, one RBAC model for visibility and policy within the Illumio Platform, one audit trail for every policy change and enforcement action across the hybrid multi-cloud, and one versioning system for policy lifecycle. Teams don't manage separate permission models, separate audit logs, or separate APIs for their Kubernetes environment.

- **Existing policy leverage** — organizations that have already invested in Illumio policy can extend it to Kubernetes without starting over.
- **Consistent enforcement model** — the same visibility → enforcement lifecycle applies everywhere, reducing risk during rollout.

1. Platform Alignment: Guiding Principles

1.1 Guiding Principle

The Agentless Kubernetes Segmentation solution should align with the Illumio Platform so that Kubernetes workloads are managed through the same workflows, data models, and interfaces that customers already use for their other workloads. From a PCE perspective, Kubernetes workloads with mapped labels should participate in the same policy, traffic, and inventory experiences as all other managed workloads.

The solution should leverage K8s native labels and Illumio labels together, applying each where it is most appropriate. Policy is authored in the Illumio Platform with the existing Illumio Policy authoring model using both – K8s labels as well as Illumio labels, and enforcement is handled by the respective enforcement points – i.e. the CNI via the operator for K8s workloads, and SGs/VENs for the non-K8s workloads.

This alignment ensures that customers adopting Kubernetes segmentation don't need to learn a new product, manage a separate policy system, or lose visibility across their hybrid environment. It also ensures the ability to segment infrastructure using the combined power of K8s native labels and Illumio labels — each applied where it delivers the most value.

1.2 Platform Alignment Requirements

REQ-ARCH-01: Unified Policy with Hybrid Enforcement

The PCE should support policies that span multiple workload types. A policy targeting workloads with specific labels (e.g., `App=Payment`, `Env=Production`) should apply to both Kubernetes workloads and non-K8s workloads that carry the same labels, without requiring the policy to be authored separately for each environment.

When a policy spans K8s and non-K8s workloads, enforcement is split between the appropriate mechanisms:

- The K8s portion is translated into a native Kubernetes NetworkPolicy and enforced by the CNI

- The non-K8s portion (e.g., an on-premises server or cloud instance) is enforced through enforcement points such as the iptables via the VEN for private-dc or SG/NSG in public cloud.

The PCE is responsible for computing the full policy intent and producing the correct enforcement artifact for each side. It does not push NetworkPolicies to the CNI directly — it generates the K8s NetworkPolicy definition and delivers it to the cluster through the selected deployment workflow (Illumio Operator, GitOps, or manual YAML export). The CNI then enforces it natively.

REQ-ARCH-02: Consistent Workload Representation

Agentless K8s workloads should be represented in the platform as workload objects — the same object type used for other managed workloads. They carry labels (mapped from K8s labels) and appear in Traffic views, Inventory, and the Map alongside all other workloads.

The Illumio platform's role for agentless container workloads is focused on visibility, policy authoring, and hybrid policy coordination:

Platform responsibilities (agentless containers):

- Representing K8s workloads with labels in the workload inventory
- Displaying K8s traffic flows in the Map and traffic query views
- Providing the UI for policy authoring, impact analysis, and versioning
- Computing hybrid policies where K8s workloads communicate with PCE-managed workloads
- Generating K8s NetworkPolicy YAML as an enforcement artifact for delivery to clusters
- Enforcing the non-K8s side of hybrid policies on PCE-managed workloads

In-cluster components:

- The agentless solution does not deploy agents (C-VEN) on K8s nodes
- An Illumio Operator is deployed at the cluster level to facilitate policy delivery, workload discovery, and communication between the Illumio and K8s cluster
- The Operator is a lightweight, non-privileged component — it does not enforce policy directly

When hybrid enforcement applies:

- In scenarios where a K8s policy includes a PCE-managed workload (on-premises server, cloud instance) as a source or destination. The PCE enforces its side of the rule on the managed workload, while the K8s side is enforced by the CNI via the generated NetworkPolicy.

REQ-ARCH-03: Unified Map with K8s Resource Hierarchy

K8s workloads should appear on the same Map as all other workloads. The Map should follow the existing Kubernetes resource hierarchy as currently implemented in the CloudSecure K8s Map:

Cluster-level hierarchy:

Cloud → Region → Account → K8s Cluster

Within a cluster (cluster-scoped resources):

- K8s Node
- K8s Custom Resource

Within a namespace (namespace-scoped resources):

- K8s Deployment
- K8s StatefulSet
- K8s DaemonSet
- K8s Service (exposes workloads as network endpoints for load balancing and discovery)
- K8s NetworkPolicy
- K8s ConfigMap
- K8s Secret
- K8s ServiceAccount (identity for processes running in pods — used for authentication and RBAC, distinct from Service)

Note: K8s Nodes are cluster-scoped, not namespace-scoped — they should appear at the cluster level in the hierarchy, not inside namespaces.

Traffic flow examples from segmentation use cases:

- **K8s Namespace → K8s Namespace** (same cluster): frontend-ns → backend-ns
- **K8s Namespace → K8s Service** (cross-cluster): payment-service in Cluster A → ledger-service in Cluster B
- **K8s Namespace → Cloud Resource**: payments namespace → AWS RDS instance, decorated with cloud metadata (VPC, region)
- **K8s Namespace → On-Premises Server**: K8s microservice → legacy database managed by Illumio
- **K8s Workload → K8s API Server**: showing API server access patterns for isolation use cases

The K8s hierarchy should be visually distinct on the map for clarity, but selection, zooming, tooltips, and all interactions should behave identically to how they work for other workload types.

REQ-ARCH-04: Existing Rulesets Apply to K8s Workloads

Illumio uses a label-based policy model where security rules are written against label dimensions (Role, Application, Environment, Location) rather than against specific workloads or IP addresses. Rules are organized into **rulesets**, and each ruleset has a **scope** — a set of label criteria that determines which workloads the rules apply to. For example, a ruleset scoped to **App=Payment, Env=Production** applies to every workload carrying those labels, regardless of whether it's a VM, cloud instance, or container.

When K8s workloads are mapped into the same label space (via the label mapping mechanism described in Section 2), they should automatically fall within the scope of existing rulesets if their labels match. This means:

- A security admin who has already built policy for **App=Payment, Env=Production** should not need to rebuild that policy for K8s workloads that carry the same labels
- The platform should compute the appropriate enforcement for each workload type — K8s NetworkPolicy for K8s workloads, existing enforcement for non-K8s workloads
- New K8s workloads that get labeled and fall within an existing ruleset's scope should be covered automatically

REQ-ARCH-05: Enforcement Lifecycle

The Illumio platform supports a phased approach to enforcement where workloads progress through stages. This allows teams to observe traffic patterns before blocking anything, reducing the risk of disrupting production:

- **Visibility Only** — traffic data is collected and displayed in the Map and traffic views, but no enforcement rules are applied. The workload is observed, not controlled.
- **Selective Enforcement** — enforcement rules are applied for specific workloads or scopes, while others remain in visibility mode. This allows incremental, low-risk rollout.
- **Full Enforcement** — all workloads in scope have active enforcement rules applied.

This lifecycle should extend to agentless K8s workloads where technically feasible:

- A K8s namespace in "Visibility Only" would collect traffic data but not apply NetworkPolicies
- A K8s namespace in "Full Enforcement" would have active NetworkPolicies enforced by the CNI

Important considerations for K8s:

Kubernetes NetworkPolicy operates on an additive allow model — there is no native "deny" rule in the standard K8s NetworkPolicy spec. A deny-all posture is achieved by creating a NetworkPolicy with an empty `podSelector: {}` and no ingress/egress rules, which implicitly blocks all traffic not explicitly allowed by other policies.

Cilium extends this model with CiliumNetworkPolicy and CiliumClusterwideNetworkPolicy resources, which support explicit deny rules and cluster-wide scope.

The enforcement lifecycle design should account for these K8s-native and CNI-specific behaviors. Where the platform's existing enforcement stages cannot be directly mapped to K8s constructs, the design should define how they are adapted or where alternatives are needed.

REQ-ARCH-06: Traffic Decoration and Visibility

Traffic from agentless K8s workloads must appear in traffic query views using the same query model as all other traffic. Traffic must be decorated with Illumio labels so it is queryable and groupable using the same label dimensions as traffic from other managed workloads. Decorated K8s traffic flows must be written to the unified decorated flow table in the single home region data store (Kusto), alongside flows from all other workload types — not stored in a separate pipeline or data store. This is a foundational requirement as Map visualization, policy computation, posture analysis, and impact simulation all depend on it.

What traffic decoration should include:

- Illumio labels (Role, App, Env, Loc) — mapped from K8s labels via the label mapping chain
- K8s-native metadata: namespace, cluster name, deployment name
- Cloud metadata (where applicable): VPC, region, cloud account, subnet
- Protocol, port, byte count, timestamp

Traffic decoration by use case:

Namespace → Namespace (same cluster)

- Source: K8s labels + Illumio labels + namespace + cluster
- Destination: K8s labels + Illumio labels + namespace + cluster

Namespace → Cloud Resource

- Source: K8s labels + Illumio labels + namespace
- Destination: Cloud resource name + VPC + region + Illumio labels (if mapped)

Namespace → On-Premises Server

- Source: K8s labels + Illumio labels + namespace + cluster
- Destination: Illumio labels + hostname + OS + location

Cross-cluster K8s → K8s

- Source: K8s labels + Illumio labels + namespace + Cluster A
- Destination: K8s labels + Illumio labels + namespace + Cluster B

What exists today (agent-based Core for Kubernetes):

- Kubelink decorates flows with workload IDs based on pod IP resolution
- Flows appear in Illumination with Illumio labels assigned via Container Workload Profiles, annotations, or LabelMap CRD
- K8s-native labels (app, version, etc.) are not directly visible in traffic queries — only Illumio labels are shown

What is needed for agentless:

- Traffic decorated with both K8s-native labels AND Illumio labels
- K8s-native metadata (namespace, cluster, deployment) as queryable fields
- Cloud metadata decoration for K8s-to-cloud traffic flows
- Decoration happens platform-side using K8s API metadata + label mapping rules

REQ-ARCH-07: Policy Lifecycle and Delivery

The platform should support the full policy lifecycle for K8s workloads — from authoring through delivery and status tracking.

Step 1 — Author

User creates policy in the Illumio UI, using K8s labels, Illumio labels, or both. The UI provides guided authoring with dynamic resource discovery from connected clusters.

Step 2 — Compute

The PCE computes the policy intent and generates the appropriate artifacts: - For K8s-only policies → K8s NetworkPolicy YAML - For hybrid policies (K8s + non-K8s) → K8s NetworkPolicy YAML for the K8s side, and enforcement rules for the non-K8s side

Step 3 — Deliver

K8s NetworkPolicy YAML is delivered to the cluster via the selected workflow: - Illumio Operator → CNI enforcement - GitOps (ArgoCD/Flux) → Git commit → sync → K8s API - Manual YAML export → kubectl apply

Non-K8s enforcement rules – no changes from current approach.

Step 4 — Track

The platform tracks policy status, versioning, drift detection, and compliance across all delivery methods. The Illumio Operator monitors the live state of NetworkPolicies in the cluster and notifies the platform of any changes — including external modifications, deletions, or drift from the desired state.

REQ-ARCH-08: Unified Platform Operations

The agentless K8s solution must share the same operational infrastructure as the rest of the Illumio platform — not introduce parallel systems for API access, permissions, auditing, or versioning.

- Single API — K8s policy authoring, workload queries, and enforcement actions must be accessible through the same API that serves data center, cloud, and endpoint operations. No separate "K8s API" or "Container API."
- Single RBAC Model — access control for K8s visibility and policy must use the same role-based access model as the rest of the platform. A scoped role that grants access to App=Payment, Env=Production applies uniformly — the user sees and manages those workloads whether they are VMs, cloud instances, or K8s deployments.
- Single Audit Trail — every K8s policy change, enforcement action, label modification, and provisioning event must be logged in the same audit system as all other platform events. One audit trail across the hybrid multi-cloud environment.
- Single Versioning System — Policy versioning, change history, diff views, and rollback must use the same versioning mechanism as other policy types.

REQ-ARCH-09: UI Changes to Align Agentless K8s with Platform Workflows

The following existing UI pages and components require updates to support agentless K8s workloads within the Illumio platform workflow:

Map

- Integrate K8s resource hierarchy (Cloud → Region → Account → K8s Cluster → K8s Namespace → resources) following the existing CloudSecure K8s Map pattern
- Render cross-boundary traffic flows (K8s ↔ non-K8s)

Traffic Query / Explorer

- Add filter keys: K8s Namespace, K8s Cluster, K8s Label (key=value), K8s Object Type
- Add columns to results: Namespace, Cluster, K8s Labels
- Support queries that span K8s and non-K8s workloads in the same result set

Inventory / Workloads

- Extend Workload object to include K8s attributes (Namespace, Cluster, Deployment, K8s Labels, Replica Count)
- Add filter keys: K8s Namespace, K8s Cluster, K8s Object Type, K8s Label
- Existing Group By must support K8s workloads (group by Application, Environment, Namespace, Cluster)

Policy / Rulesets

- The existing policy authoring UI and mechanism should be extended to support K8s workloads — customers author rules using Illumio labels or K8s labels
- Policy authoring follows the current Illumio policy model. Policy authoring must NOT support authoring K8s network policy, including namespaceselector / podselectors.
- Add CNI compatibility indicator per rule

Policy List (All Policies)

- The Policy List page should include policies intended to cover Kubernetes' clusters and namespaces, and it should support the same columns as existing policies.

Standard Filter Bar (global)

New filter keys to add across all pages: - **K8s Cluster** — filter by cluster name - **K8s Namespace** — filter by namespace - **K8s Label** — filter by any K8s label key=value pair - **K8s Object Type** — filter by Deployment, StatefulSet, DaemonSet, Service - **K8s Resource Name** — filter by workload/deployment name

These filters should behave identically to existing filters and work uniformly across K8s and non-K8s workloads.

1.3 Agent-Based vs. Agentless: What Changes

Today, Illumio Core for Kubernetes uses C-VEN (an agent on every node) and Kubelink (a pod per cluster) to collect traffic, map workloads, and enforce policy. The agentless approach replaces both with a lightweight Illumio Operator at the cluster level and shifts enforcement to the CNI.

Workload Discovery

- Agent-based: Kubelink queries the K8s API from inside the cluster, reports workloads to the platform
- Agentless: The Illumio Operator and platform query the K8s API — no per-node agents needed

Label Mapping (K8s Labels → Illumio Labels)

- Agent-based: LabelMap CRD deployed in the cluster via Helm Chart, processed by Kubelink
- Agentless: Label mapping rules configured in the platform — no CRD deployed in the cluster

Traffic Flow Collection

- Agent-based: C-VEN captures flows on each node
- Agentless: CNI provides flow data natively (Cilium Hubble, Calico flow logs, cloud VPC Flow Logs)

Traffic Decoration

- Agent-based: Kubelink resolves pod IPs to workload IDs, adds Illumio labels to flow records
- Agentless: The platform resolves pod IPs using K8s API metadata and applies labels via mapping rules

Policy Delivery

- Agent-based: Platform → Kubelink (CLAS cache) → C-VEN programs iptables/nftables
- Agentless: Platform generates NetworkPolicy YAML → Illumio Operator delivers to cluster, or via GitOps/manual export

Policy Enforcement

- Agent-based: C-VEN programs iptables/nftables rules per workload
- Agentless: The installed CNI enforces K8s NetworkPolicy natively

In-Cluster Footprint

- Agent-based: C-VEN DaemonSet (one pod per node) + Kubelink Deployment (one pod per cluster)
- Agentless: Illumio Operator only (one deployment per cluster) — no per-node agents

The platform takes on the work that Kubelink used to do (discovery, decoration, label mapping), while the CNI takes on the work that C-VEN used to do (enforcement). The platform's role as the policy engine and visualization layer remains the same.

2. Label Architecture: K8s Labels and Illumio Labels

2.1 The Problem

Illumio uses a multidimensional labeling system (Role, Application, Environment, Location) to classify workloads and author policy. Kubernetes uses its own label system (`app`, `version`, `app.kubernetes.io/name`, `app.kubernetes.io/component`, etc.) that is namespaced to each cluster and has no inherent relationship to Illumio's label dimensions.

Today, the Core for Kubernetes (C-VEN + Kubelink) solution bridges this gap using three mechanisms:

1. **Container Workload Profiles** — assign Illumio labels at the namespace level to all pods in that namespace
2. **Annotations** — `com.illumio.role`, `com.illumio.app`, `com.illumio.env`, `com.illumio.loc` on deployment manifests
3. **LabelMap CRD** — maps K8s label keys/values to Illumio label types/values via a Custom Resource Definition deployed in the cluster

Current LabelMap CRD: How It Works Today

The LabelMap CRD (kind: `LabelMap`, apiVersion: `ic4k.illumio.com/v1alpha1`) defines mappings using `fromKey` (K8s label) → `toKey` (Illumio label type), with optional `valuesMap` for value translation and `allowCreate` for auto-creating labels.

Workload label mapping example (from Core for Kubernetes documentation):

```
kind: LabelMap
apiVersion: ic4k.illumio.com/v1alpha1
metadata:
  name: default
workloadLabelMap:
  - fromKey: app      # K8s label key
    toKey: application # Illumio label type
    allowCreate: true
  - fromKey: pod-type # K8s label key
```

```
toKey: role          # Illumio label type
```

```
valuesMap:
```

```
- from: lb           # K8s value
```

```
to: Load Balancer   # Illumio value
```

```
- from: db
```

```
to: Database
```

```
- fromKey: environ
```

```
toKey: env
```

```
allowCreate: false
```

```
valuesMap:
```

```
- from: EKS
```

```
to: AmazonK8SService
```

```
- fromKey: stage
```

```
toKey: role
```

```
allowCreate: true
```

Node label mapping example:

```
nodeLabelMap:
```

```
- allowCreate: true
```

```
fromKey: topology.kubernetes.io/region # Standard K8s node label
```

```
toKey: loc          # Illumio Location
```

```
valuesMap:
```

```
- from: eu-west-1
```

```
to: Amazon
```

```
- from: eu-west-2
```

```
to: Amazon
```

```
- allowCreate: true
```

```
fromKey: node-type
```

toKey: k8s-node

Label assignment precedence (highest to lowest):

PRIORITY	SOURCE	APPLIES TO
1	Container Workload Profile	Namespace-level labels → all workloads in namespace
2	LabelMap CRD	K8s labels matched via mapping rules
3	Annotations (<code>com.illumio.*</code>)	Per-deployment/pod Illumio annotations

Mapping capabilities:

- Map K8s label type + value → Illumio label type + value (e.g., `app: CRM` → `application: CRM-0531`)
- Map K8s label type only → Illumio label type (e.g., `app` → `application`, values pass through)
- Auto-create Illumio labels that don't exist (`allowCreate: true`)
- Label source attribution visible in UI (`com.ilo-result.<label_type>` annotation shows whether label came from CWP, annotation, or label-map)

Limitation: LabelMap CRD is only supported in CLAS mode (Kubelink with Cluster Local Actor Store). It requires in-cluster deployment via Helm Chart.

The agentless solution must provide equivalent label bridging without deploying agents, Kubelink, or CRDs into the cluster.

2.2 Requirements

REQ-LABEL-01: Dual Label Model Support

The platform must support Kubernetes workloads being represented using both native K8s labels and Illumio labels simultaneously. Illumio labels remain the basis for policy computation and Map visualization.

REQ-LABEL-02: Agentless Label Mapping

The solution must provide a mechanism to map K8s labels to Illumio labels without deploying agents or CRDs into the cluster. Options include:

API-driven mapping — the platform reads K8s labels via the K8s API and applies mapping rules defined in the platform

Platform-side LabelMap — equivalent to the LabelMap CRD but configured in the platform console/API, not as a K8s resource

Auto-discovery mapping — the platform discovers common K8s label patterns and suggests Illumio label mappings

REQ-LABEL-03: Label Mapping Precedence (Agentless)

When multiple label sources exist, the following precedence must be maintained (highest to lowest):

1. Platform-defined label assignments (equivalent to Container Workload Profiles)
2. K8s-to-Illumio label mapping rules (equivalent to LabelMap CRD)
3. K8s annotations matching `com.illumio.*` pattern
4. Raw K8s labels (displayed as-is, used for K8s-native policy authoring)

REQ-LABEL-04: Namespace Label Decoration

Kubernetes namespaces must be decoratable with Illumio labels (Environment, Location, Application) to enable:

- Traffic flows from workloads in that namespace to appear in the Map with correct context
- Policy scoping using namespace-level label assignments
- Automatic inheritance — workloads in a labeled namespace inherit the namespace's labels unless overridden by workload-level mapping

REQ-LABEL-05: K8s Labels in the Platform UI

The platform's Map, traffic views, and Inventory are built around Illumio labels (Role, App, Env, Loc). K8s-native labels do not natively fit into this model without mapping.

Primary path: K8s labels should be mapped to Illumio labels so that K8s workloads are represented using the same label dimensions as all other workloads. This enables full platform parity — policy computation, traffic grouping, and Map visualization all work through Illumio labels.

Supplementary K8s filters: Where K8s metadata does not map to Illumio label dimensions (e.g., namespace name, cluster name, object type), the platform should add supplementary filter keys:

- K8s Namespace
- K8s Cluster
- K8s Object Type (Deployment, StatefulSet, DaemonSet, Service)
- K8s Resource Name

In practice:

- To see a K8s workload in the Map grouped by Application, the workload's K8s `app` label must be mapped to an Illumio `application` label
- To filter workloads by namespace, the K8s Namespace filter is used (no label mapping needed)
- K8s labels that are not mapped will not appear in Map grouping or policy computation — they remain visible as raw metadata in the workload detail view

REQ-LABEL-06: Label Source Attribution

For any workload displayed in the UI, the platform must indicate the source of each label:

- `platform-assignment` — assigned via platform configuration
- `label-map` — derived from K8s label via mapping rule
- `annotation` — from K8s manifest annotation
- `k8s-native` — raw K8s label, no mapping applied

3. Traffic Decoration and Visualization

3.1 The Problem

In Core for Kubernetes, Kubelink intercepts traffic flow reports from C-VEs, "decorates" them with workload IDs based on IP addresses, and sends this information to the platform. The platform then visualizes these flows using Illumio labels.

The agentless solution must achieve equivalent traffic decoration using K8s API and CNI-level flow data, without C-VE or Kubelink.

3.2 Requirements

REQ-TRAFFIC-01: Agentless Flow Collection

Traffic flows must be collected from Kubernetes clusters without per-node agents.

- The primary collection method is CNI-native flow telemetry (e.g., Cilium Hubble, Calico flow logs), which is already available in supported CNI environments.
- For traffic that crosses the cluster boundary to cloud resources, cloud provider flow logs (VPC Flow Logs, NSG Flow Logs) may be used as a supplementary source.

REQ-TRAFFIC-02: Flow Decoration with Illumio Labels

Collected traffic flows must be decorated with Illumio labels during ingestion. Decoration must resolve:

- Source pod IP → Source workload → Illumio labels (via mapping chain)
- Destination pod IP → Destination workload → Illumio labels
- Destination service (ClusterIP) → K8s Service → Illumio labels
- External IP → IP List / FQDN list / Cloud resource mapping / Private-DC

REQ-TRAFFIC-03: Traffic Display in the Map

Decorated traffic must appear in the Map (the same Map, not a separate view) with:

- Same visual treatment as traffic from other managed workloads
- Grouping by Illumio labels AND by K8s metadata (namespace, cluster) — both available in the Group By dropdown
- Color coding: green for policy-allowed traffic, red for policy-denied/blocked, amber for unprotected/default-allow
- Cross-boundary traffic (K8s ↔ non-K8s) must render as a single flow line
- Draft View must show the impact of staged K8s Network Policies

K8s Cluster Visualization in the Unified Map

- The Map should support a grouping mode (e.g., “Data Center Type” or similar) where Kubernetes (K8s) clusters are displayed within their own distinct visual boundary and follow a Kubernetes-native hierarchy expansion model (Cluster → Namespace →

Deployments/Services). This enables Kubernetes resources to remain visually distinct while still being part of the same unified Map experience.

- The Group By controls for the Kubernetes cluster portion of the Map must be configurable independently from the Group By applied to the rest of the Map. For example, the non-Kubernetes portion of the Map may be grouped by Application and Environment, while the Kubernetes portion may simultaneously be grouped by Namespace and Deployment, without requiring the user to switch the grouping model for the entire Map.
- This dual-grouping capability ensures that Kubernetes-native navigation patterns (cluster → namespace → workloads) coexist with Illumio label-based grouping (application → environment → role) on the same canvas, without forcing a single unified grouping model across both domains.

REQ-TRAFFIC-04: Policy Advisor for K8s Workloads

The existing Policy Advisor capability should be extended to support agentless K8s workloads. Policy Advisor auto-creates policy recommendations based on observed traffic using Application and Environment labels. For K8s workloads, Policy Advisor should additionally support namespace-based policy recommendations.

Policy Advisor must generate recommendations for the following K8s scenarios:

- Namespace to Namespace — recommend allow rules between K8s namespaces within the same cluster based on observed inter-namespace traffic patterns
- Inbound/Outbound to non-K8s (Application-Environment) — recommend allow rules between K8s workloads and non-K8s workloads (VMs, cloud instances, on-premises servers) using Application and Environment labels, based on observed cross-environment traffic
- Inbound/Outbound to non-K8s (IP-based) — recommend allow rules between K8s workloads and external destinations identified by IP address or CIDR range, for traffic that cannot be resolved to labeled workloads
- Recommendations should be based on selectable observation windows (1 hour, 6 hours, 24 hours, 3 days, 7 days, 30 days) and include coverage scoring — the percentage of observed traffic that is already covered by existing policies

REQ-TRAFFIC-05: Protected vs. Exposed Traffic Visualization

The Map must visually differentiate:

- **Protected traffic** — governed by an active NetworkPolicy
- **Exposed traffic** — flowing by default (no policy applied, K8s allow-all)
- **Blocked traffic** — denied by an active policy

This enables security teams to prioritize where new rules are needed.

4. Inventory: K8s Workloads in the Platform

4.1 The Problem

Illumio represents workloads as objects with Illumio labels. K8s workloads (Deployments, StatefulSets, DaemonSets, Services) have a different metadata model. The agentless solution must represent K8s workloads alongside traditional workloads without creating separate K8s-only UI patterns.

4.2 Requirements

REQ-INV-01: K8s Workloads as Standard Workload Objects

The Workload data object must be extended to include K8s-specific attributes as standard metadata fields:

- Namespace
- Cluster (name, provider, region)
- K8s Object Type (Deployment, StatefulSet, DaemonSet, Service)
- K8s Labels (full set)
- Replica count
- Container images
- Service account

K8s resources (Deployments, Services, Namespaces, NetworkPolicies, etc.) should be kept as a separate, dedicated subsection within each respective inventory section to maintain clarity and allow K8s-specific navigation without mixing them into the general workload list.

REQ-INV-02: Unified Inventory View

K8s workloads must appear in the same inventory views as cloud instances and on-premises servers. No separate K8s inventory tab or section. A query for "all workloads in App=Payment" must return platform-managed servers AND agentless K8s Deployments if they carry the same Illumio application label.

REQ-INV-03: Workload Detail View

Clicking a K8s workload must show:

- Illumio labels (if mapped)
- K8s labels (always)
- Label source attribution (per REQ-LABEL-06)
- Namespace and cluster context
- Associated policies (K8s NetworkPolicies and Illumio policies)
- Traffic flows (ingress/egress)
- Enforcement status

5. Policy Authoring

5.1 The Problem

Illumio policy uses Illumio labels (Role/App/Env/Loc) to define source and destination in rules. K8s labels are also allowed to be used in Illumio labels.

5.2 Requirements

REQ-POL-01: Dynamic Resource Discovery

The policy authoring UI must dynamically discover and expose only relevant resources:

- K8s labels (from live cluster metadata)
- Namespaces (across all connected clusters)
- Clusters (with CNI type and version)
- Deployments and workload types
- Illumio labels (from the platform)

REQ-POL-02: Ports and Protocols

- Protocol selector: TCP, UDP, SCTP
- Port number input with named port support (http/80, https/443, postgresql/5432, etc.)
- Multi-port support within a single rule

- "All Ports" toggle with warning

REQ-POL-03: Policy Preview and Validation

- Basic validation (syntax, required fields, CNI compatibility)
- CNI compatibility indicator (warn when using Cilium-only or Calico-only features)

REQ-POL-04: Impact Simulation

Combination of draft views in the traffic table / map, and the "Show impact" link simulates Impact.

REQ-POL-05: Policy Output Formats

- Download as K8s NetworkPolicy YAML (kubectl apply compatible)
- Save as Illumio policy definition (for Illumio-extended constructs)
- Export for GitOps (commit-ready format)

REQ-POL-09: Segmentation Use Cases

Must support the following authoring scenarios:

USE CASE	SOURCE	DESTINATION
Same-cluster namespace isolation	Namespace A (Cluster 1)	Namespace B (Cluster 1)
Cross-cluster service access	Namespace (Cluster 1)	K8s Service (Cluster 2)
K8s to cloud resource	Namespace / Pod label	Cloud resource (RDS, S3, SQL) via CIDR/tags

USE CASE	SOURCE	DESTINATION
K8s to Illumio-labeled workload	Namespace / Pod label	On-premises server with Illumio labels
Cluster-wide guardrail	All pods	Deny Internet egress by default
Pod-label + namespace combo	Namespace + specific pod label	Target service/namespace
API server isolation	Specific pods only	K8s API server (port 6443)

6. Network Policy Posture and Advisory

6.1 Requirements

REQ-POSTURE-01: Existing Policy Visibility

The platform must discover and display all existing K8s NetworkPolicies in connected clusters:

- Policy name, namespace, cluster
- Selectors (podSelector, namespaceSelector)
- Ingress/egress rules summary
- Whether the policy was created by Illumio or externally (kubectl, Helm, ArgoCD, Terraform)
- Source attribution: **Illumio-managed** vs **External**

REQ-POSTURE-02: Policy Coverage Analysis

For each namespace/cluster, the platform must compute and display:

- Coverage score (percentage of workloads governed by at least one policy)
- Protected vs. exposed workload count
- Namespaces with no policies (fully exposed)
- Workloads with overly permissive rules (e.g., allow-all egress)

REQ-POSTURE-03: Policy Findings and Recommendations

The advisory system must detect and surface:

- **Missing isolation** — namespace with no default deny policy
- **Unrestricted egress** — rules allowing 0.0.0.0/0
- **Stale policies** — policies with no matching traffic in observed window
- **Over-restrictive policies** — rules blocking observed legitimate traffic
- **Overlapping policies** — multiple policies with conflicting selectors
- **Missing DNS exception** — deny-all egress without kube-dns rule

Each finding must include severity (High/Medium/Low), affected namespace/cluster, and a suggested remediation action.

REQ-POSTURE-04: Guardrail Compliance

Before provisioning, the platform must perform a pre-flight guardrail check:

- Validate the policy against organizational guardrails (e.g., "no unrestricted egress", "default deny required")
- If a guardrail violation is detected, the policy must not be enforceable until resolved or explicitly overridden
- Guardrail conflict status must be displayed in the policy list and detail views
- Override requires explicit acknowledgment and is logged for audit

Note: The list of Guardrail policies will be provided separately; this requirement reflects the intended future use of Guardrail policies.

7. Policy Versioning and Lifecycle

7.1 Requirements

REQ-LIFE-01: Policy States

Every policy must have a visible lifecycle state:

STATE	DEFINITION
Drafting	Policy is being authored. Not enforced. Can be edited freely.
Pending Review	Policy has been submitted for approval.
Approved	Reviewed and approved. Ready to deploy.
Deploying	Being pushed to cluster via selected workflow.
Active/Enforced	Live and actively enforcing on cluster traffic.
Failed/Out-of-Sync	Deployment failed or live state has drifted from desired state.

REQ-LIFE-02: Versioning and Change History

- Every policy change creates a new version
- Full change history with timestamp, author, and diff view
- Ability to compare any two versions side-by-side

REQ-LIFE-03: Rollback

- Rollback to any previous version with one action
- Rollback must re-deliver the previous version through the same deployment workflow
- Rollback preserves the rolled-back version in history (doesn't delete)

REQ-LIFE-04: Deployment Workflows

The platform must support the following enforcement workflows (priority order):

1. **Manual YAML Export** — UI authoring → Download .yaml → kubectl apply
2. **Illumio Operator** — UI authoring → Illumio Operator → CNI enforcement
3. **Pure GitOps** — UI authoring → Git commit → ArgoCD/Flux sync → K8s API
4. **Illumio + GitOps** — UI authoring → Git (source of truth) → ArgoCD → CNI

REQ-LIFE-05: Drift Detection

- Visual indicator when live cluster state differs from platform-defined policy
- First iteration: detection and display only
- Second iteration: "Auto-Remediate" option to re-apply desired state

REQ-LIFE-06: Enforcement Level Indicators

Each cluster must display its enforcement capability:

- **Cilium Full** — full L3/L4/L7 via Cilium
- **Native Only** — standard K8s NetworkPolicy via any CNI
- **Hybrid with NSG** — K8s NetworkPolicy + cloud security groups
- **Calico Full** — Calico-specific extensions available

8. Enforcement (Agentless)

8.1 Requirements

REQ-ENF-01: Dynamic Policy Updates for Environment Changes

- Policy enforcement must remain current as the underlying environment changes. When workload attributes — such as IP addresses, labels, or group membership — change on either side of a policy, the enforcement artifacts must be updated automatically to reflect the new state.
- This applies to any environment change that affects policy resolution: IP address changes, label reassignments, workload scaling, new workloads entering a policy scope, or workloads being decommissioned. The Illumio Operator must detect these changes and update the corresponding NetworkPolicy objects in the cluster without manual intervention.

REQ-ENF-02: CNI-Native Enforcement

Rely on the installed CNI to enforce NetworkPolicies:

- AWS VPC CNI
- GKE Dataplane V2
- Azure CNI (Powered by Cilium)
- Cilium
- OpenShift OVN
- Calico

REQ-ENF-03: Enforcement SLA

NetworkPolicy must be enforced within 5 minutes after the user applies the policy. The NetworkPolicy object must be created and associated with the correct pods within this window.

REQ-ENF-04: Multi-CNI Compatibility

Policies must work consistently across clusters running different CNIs. The UI must clearly indicate when a policy uses CNI-specific features (e.g., CiliumClusterwideNetworkPolicy).

REQ-ENF-05: Multi-Cluster Distribution

A single policy authored in the UI must be distributable across multiple clusters. The platform handles per-cluster translation and delivery.

REQ-ENF-06: Enforcement Status

Per-policy, per-cluster enforcement status:

- **Applied** — successfully pushed and active

- **Pending** — not yet provisioned
- **Error** — provisioning failed (conflict, CNI issue, API error)
- **Approved / Not Approved** — review gate status
- **Guardrail Conflict** — blocked by organizational guardrail

REQ-ENF-07: Blocked Flow Visibility

Blocked/denied flows must appear in the UI with:

- Source and destination context (workload, namespace, cluster)
- Matched rule name and policy name
- Protocol/port

9. Summary: Platform Alignment at a Glance

9.1 Platform Capabilities: Shared Across Both Approaches

These capabilities work the same way regardless of whether the workload is agent-based or agentless.

Policy Engine — The platform computes policy for K8s workloads using the same engine it uses for other managed workloads. One computation, one policy intent.

Workload Representation — K8s workloads are workload objects — same type, same schema, same label model. They appear in Inventory, Map, and traffic views alongside all other workloads.

Map — One Map. K8s workloads appear on the same canvas. The K8s hierarchy (Cloud → Region → Account → Cluster → Namespace → resources) is navigable through existing patterns.

Traffic Queries — One query interface. K8s traffic is queryable using the same filters.

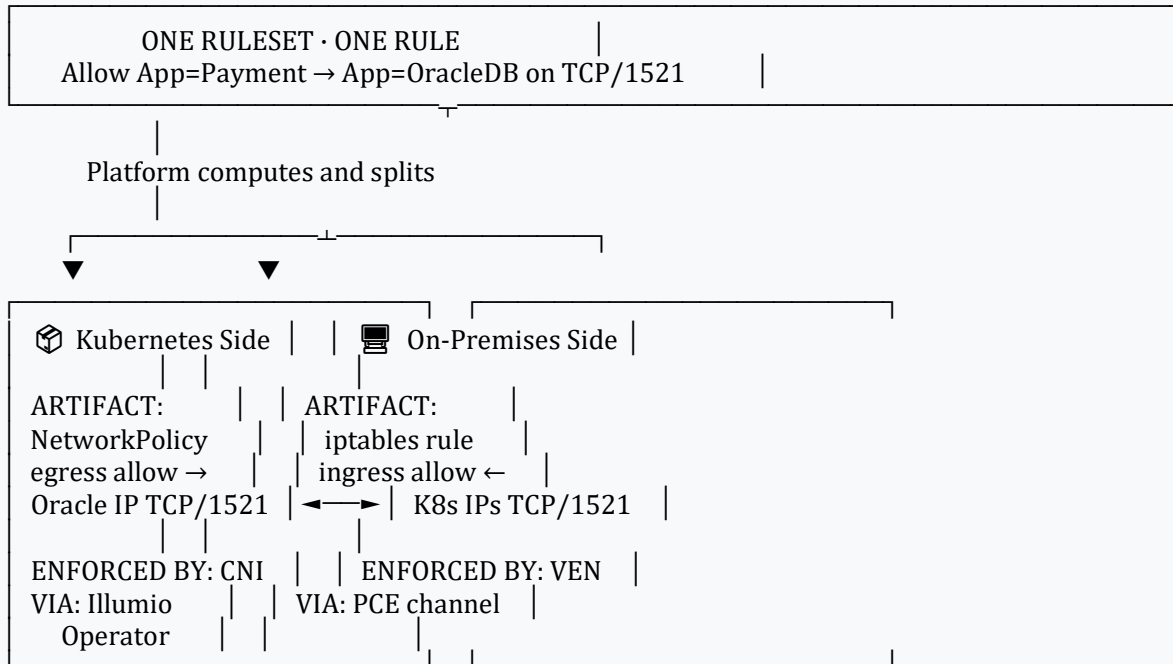
Inventory — One inventory. A search by application label returns both managed servers and K8s Deployments.

Policy Lifecycle — Same states: Drafting → Pending Review → Approved → Deploying → Active/Enforced → Failed/Out-of-Sync.

9.2 Hybrid Policy: Where Both Sides Work Together

When a policy spans K8s and non-K8s workloads, the platform computes one policy intent from one ruleset, but enforcement is split across two mechanisms.

Example: Allow `App=Payment` → `App=OracleDB` on TCP/1521



What the user sees: One rule. One ruleset. One flow line in the Map.

- The user authors one rule using Illumio labels — no awareness of the enforcement split
- The platform determines that the source is a K8s workload and the destination is a VEN-managed server
- It generates the right artifact for each side and delivers them through the appropriate channel
- Traffic between the two appears as a single decorated flow in the Map